

PRÁCTICA 1

Primer proyecto: el .ioc, CubeMX y el reloj

DOCUMENTO	ASIGNATURA	TITULACIÓN	CENTRO
Del alumnado	**Arq. de Computadores II**	**Grado en Ing. Informática · 2.º**	**ESEI · Universidade de Vigo**

Contenido

1. Ficha de la práctica 3

2. Qué se construye 3

3. De dónde sale el código 4

3.1 El .ioc es un fichero de texto 4

3.2 Qué genera CubeMX y qué no 5

3.3 El contrato: los bloques USER CODE 5

4. El reloj: el dato que no está en el código 6

5. Desarrollo paso a paso 8

Paso 1. Crear el proyecto 8

Paso 2. Abrir el .ioc con un editor de texto 9

Paso 3. El LED parpadeante 9

Paso 4. El botón 9

Paso 5. Las dos esperas 10

Paso 6. Cambiar el reloj y volver a medir 11

Paso 7. Comprueba el reloj sin fiarte del código 13

6. Criterios de aceptación 13

7. Criterios de evaluación 14

8. Entregables 15

9. Uso de asistentes de inteligencia artificial 16

Anexo A. Las claves del .ioc que conviene reconocer 17

Anexo B. Qué hay en la carpeta del proyecto 18

Anexo C. Conexionado 18

1. Ficha de la práctica

CAMPO	VALOR
Sesión	Sesión 1 de 13
Duración	2 horas presenciales
Modalidad	Individual, en el laboratorio
Requisitos previos	Ninguno. Es la primera sesión
Material	Placa NUCLEO-F401RE · cable USB · STM32CubeMX · STM32CubeIDE · un cronómetro
Entregables	Proyecto CubeIDE, tabla de medidas y memoria (apartado 8)
Plazo	Una semana desde el final de la sesión, por Moovi

2. Qué se construye

Un programa muy corto: el LED verde de la placa parpadea, y el botón azul cambia **cómo se mide la espera** entre parpadeo y parpadeo.

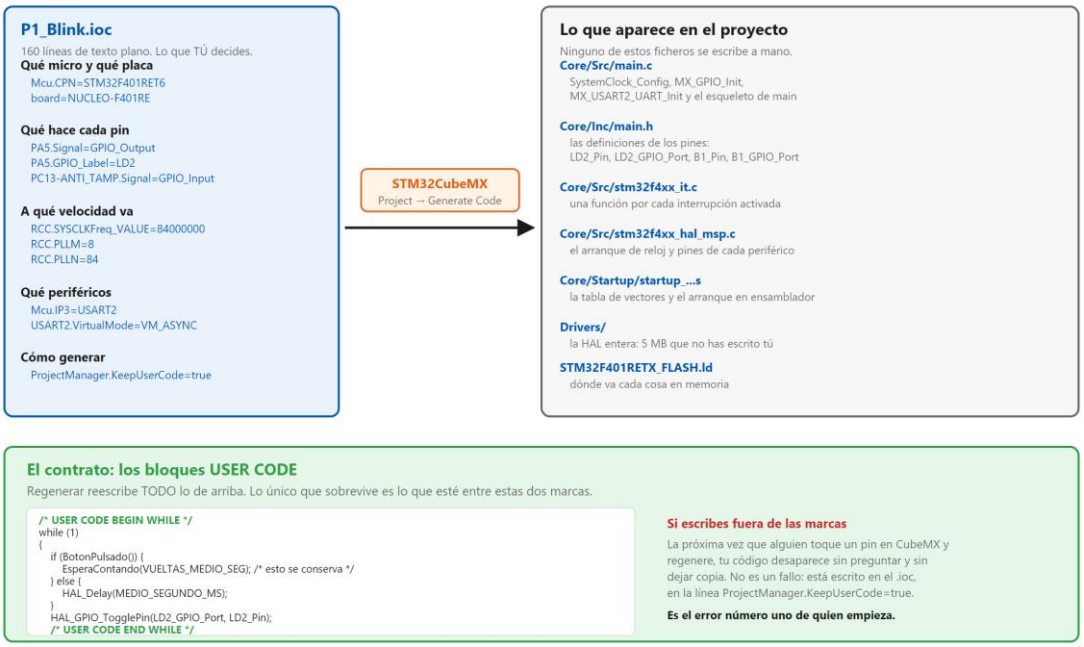
BOTÓN	CÓMO SE ESPERA	QUÉ CUENTA
Suelto	`HAL_Delay(500)`	milisegundos
Pulsado	Un bucle vacío que da muchas vueltas	vueltas de bucle

Bien ajustado, los dos parpadeos van igual. Después se cambia **una sola línea** de la configuración —la frecuencia a la que va el microcontrolador— y se vuelve a mirar: uno de los dos parpadeos sigue igual y el otro ha cambiado.

> Suena a poca cosa para dos horas, y esa es la idea. El programa es corto **a propósito**, para que quepa tiempo de mirar de dónde ha salido: quién escribió las 250 líneas de `main.c` que no has escrito tú, dónde está guardado que PA5 es una salida, y por qué el mismo código se comporta distinto según un número que no aparece en ninguna línea de C.

3. De dónde sale el código

Al crear un proyecto para la NUCLEO-F401RE aparecen de golpe unos 5 MB de ficheros. Conviene saber quién los ha puesto ahí antes de tocar nada.



EL .IOC ES LA FUENTE; EL RESTO LO GENERA CUBEMX A PARTIR DE ÉL.

3.1 El .ioc es un fichero de texto

`P1_Blink.ioc` no es un formato binario ni una base de datos: son **159 líneas de texto plano**, con la forma `clave=valor`. Se puede abrir con el Bloc de notas, y conviene hacerlo al menos una vez.

OCHO LÍNEAS DEL .IOC DE ESTA PRÁCTICA

```
Mcu.CPN=STM32F401RET6
PA5.Signal=GPIO_Output
PA5.GPIO_Label=LD2
PC13-ANTI_TAMP.Signal=GPIO_Input
PC13-ANTI_TAMP.GPIO_Label=B1
RCC.SYSCLKFreq_VALUE=84000000
ProjectManager.KeepUserCode=true
board=NUCLEO-F401RE
```

Ahí está **todo lo que has decidido**: qué micro, qué hace cada pin, a qué velocidad va y cómo se genera el proyecto. La ventana gráfica de CubeMX es una forma cómoda de editar este fichero, nada más.

3.2 Qué genera CubeMX y qué no

LO GENERA CUBEMX	LO ESCRIBES TÚ
`SystemClock_Config`, `MX_GPIO_Init`, `MX_USART2_UART_Init`	La lógica del programa, dentro del `while(1)`
Las definiciones `LD2_Pin`, `B1_GPIO_Port`… en `main.h`	Tus funciones y tus constantes
`stm32f4xx_it.c`: una función por interrupción activada	El contenido de esas funciones
`Drivers/`: la biblioteca HAL entera	Nada. No se toca
El fichero de enlazado `.ld` y el arranque en ensamblador	Nada. No se toca

3.3 El contrato: los bloques USER CODE

Repartidos por los ficheros generados hay parejas de comentarios como estas:

LA MARCA

```
/* USER CODE BEGIN 2 */

/* USER CODE END 2 */
```

Cada vez que se pulsa **Project → Generate Code**, CubeMX reescribe los ficheros generados de arriba abajo. Lo único que respeta es lo que esté **entre** un `BEGIN` y su `END`.

> **Escribir fuera de las marcas es el error número uno de quien empieza.** El código funciona, se compila, se graba en la placa; y desaparece sin previo aviso el día que alguien cambia un pin y regenera. No hay copia de seguridad ni pregunta de confirmación: está escrito en el propio `.ioc`, en la línea `ProjectManager.KeepUserCode=true`.

4. El reloj: el dato que no está en el código

El STM32F401 puede funcionar a distintas frecuencias. La que use en cada momento se decide en el `.ioc`, en la pestaña **Clock Configuration**, y de ahí sale el `SystemClock_Config` que genera CubeMX.

CONFIGURACIÓN	FRECUENCIA DEL NÚCLEO	CÓMO SE OBTIENE
Oscilador interno directo	16 MHz	El HSI tal cual, sin multiplicar
Con el multiplicador PLL	**84 MHz**	16 / 8 = 2 MHz, × 84 = 168, / 2 = 84

84 MHz es el máximo del F401 y es la configuración que usan todas las prácticas de la asignatura. Pero fíjate en que ese número **no aparece en ninguna línea del programa que escribes tú**. Está en el `.ioc`, y de rebote en una función generada que nadie suele abrir.

> De ahí sale la pregunta de la práctica: **¿qué partes de un programa cambian si cambia el reloj, y cuáles no?** Los pasos 5 y 6 la responden midiendo, no razonando.

5. Desarrollo paso a paso

Paso 1. Crear el proyecto

Desde la versión 2.0, **CubeIDE ya no trae CubeMX dentro**: un proyecto creado desde CubeIDE sale **sin `.ioc`**, y sin él esta práctica no se puede hacer. El proyecto se crea en **STM32CubeMX** y después se abre en CubeIDE:

1. En CubeMX: **File → New Project**, pestaña **Board Selector**, busca `NUCLEO-F401RE` y selecciónala. Al preguntar si se inicializan los periféricos en su modo por defecto, responde **que sí**: interesa ver qué añade el generador por su cuenta.
2. En la pestaña **Project Manager**: pon el nombre y la carpeta del proyecto, en **Toolchain / IDE** elige **STM32CubeIDE** y deja marcado **Generate Under Root**.
3. Pulsa **Generate Code**.
4. En CubeIDE: **File → Import → General → Existing Projects into Workspace**, y elige esa carpeta.

> **CubeMX propone por defecto otro entorno, EWARM, que es el de IAR.** Si no lo cambias, genera un proyecto que CubeIDE no sabe abrir, y la importación no encuentra nada. En el `.ioc` se ve en la línea `ProjectManager.TargetToolchain`, que tiene que decir `STM32CubeIDE`.
>
> **Guarda el proyecto en una carpeta sin letras acentuadas.** Si la ruta lleva alguna —una carpeta llamada «Práctica 1», o un usuario que se llame «José»— el programa compilará y se grabará en la placa sin problema, pero **al depurar CubeIDE no encontrará el `main.c`** y solo te ofrecerá ver ensamblador. El compilador guarda la ruta dentro del ejecutable en UTF-8 y el depurador la lee con la codificación de Windows: para cualquier letra acentuada las dos no coinciden. Los espacios y los paréntesis no molestan; solo las tildes y la eñe.

Paso 2. Abrir el .ioc con un editor de texto

Antes de tocar la ventana gráfica, abre el `.ioc` como texto plano y búscale estas cuatro cosas:

5. La línea que dice qué micro es.
6. Qué pines ha configurado CubeMX **sin que se lo pidieras**, solo por haber elegido la placa. Hay al menos cinco.
7. A qué frecuencia ha quedado el sistema.
8. La línea que decide si tu código se conserva al regenerar.
9. La línea que dice **para qué entorno** se genera el proyecto.

Anota las respuestas: la primera cuestión de la memoria va sobre esto.

Paso 3. El LED parpadeante

En CubeMX, configura **PA5** como `GPIO_Output` y ponle la etiqueta `LD2`. Genera el código y, **en CubeIDE, refresca el proyecto con F5**: si no, sigue viendo los ficheros de antes. Después escribe el parpadeo dentro del bucle infinito, **entre las marcas USER CODE**:

DENTRO DE USER CODE BEGIN WHILE

```
HAL_GPIO_TogglePin(LD2_GPIO_Port, LD2_Pin);
HAL_Delay(500);
```

Compila, graba y comprueba que parpadea una vez por segundo: medio segundo encendido y medio apagado.

Paso 4. El botón

En CubeMX, configura **PC13** como `GPIO_Input` con la etiqueta `B1`, genera y refresca como antes. Es el botón azul de la placa, y está cableado **activo a cero**: pulsado se lee `GPIO_PIN_RESET`.

LA LECTURA

```
if (HAL_GPIO_ReadPin(B1_GPIO_Port, B1_Pin) == GPIO_PIN_RESET) {
    /* pulsado */
```

```
}
```

> Léelo tal cual, sin filtrar nada. Un pulsador real no cambia de estado limpiamente: rebota durante unos milisegundos. Aquí da igual, porque solo se mira si está pulsado o no. En la Práctica 2 sí importará, y ahí se verá cómo se trata.

Paso 5. Las dos esperas

Las dos esperas del título son estas. **La primera ya la estás usando**: es la del Paso 3, la que viene hecha en la biblioteca. Se le pide un tiempo en milisegundos.

LA ESPERA DE LA BIBLIOTECA

```
HAL_Delay(500);   /* medio segundo */
```

La segunda hay que escribirla, y no usa la HAL para nada: espera **contando vueltas** de un bucle vacío.

LA ESPERA «A PELO»

```
static void EsperaContando(uint32_t vueltas)
{
    for (volatile uint32_t i = 0; i < vueltas; i++) {
        /* a proposito, aqui no hay nada */
    }
}
```

> El `volatile` es imprescindible. Sin él, el compilador tiene permiso para ver un bucle cuyo resultado no usa nadie y **borrarlo entero**: el programa quedaría sin espera y el LED parpadearía a una velocidad invisible.

Puedes comprobarlo, pero **hay que hacerlo en la configuración Release**, no en Debug. Y esa diferencia es parte de la lección:

CONFIGURACIÓN	OPTIMIZACIÓN	CON VOLATILE	SIN VOLATILE
Debug	`-O0`	el bucle está	**el bucle sigue estando**
Release	`-Os`	el bucle está	**el bucle desaparece**

> En Debug el compilador no optimiza nada, así que quitar `volatile` no cambia el resultado y parece que la palabra sobra. **Sobra hasta el día que compilas con optimización**, y entonces el programa hace algo distinto sin que hayas tocado una línea. Por eso `volatile` no es una recomendación de estilo: es la única forma de decirle al compilador que ese bucle importa aunque no lo parezca.

Ahora haz que el botón elija entre las dos esperas, y **ajusta el número de vueltas con el cronómetro** hasta que el parpadeo con el botón pulsado se parezca al de sin pulsar. Anota el número al que has llegado.

Paso 6. Cambiar el reloj y volver a medir

Abre el `.ioc` en CubeMX, ve a **Clock Configuration** y cambia el origen del reloj del sistema del PLL al oscilador interno directo: de 84 MHz a 16 MHz. Regenera el código, refresca en CubeIDE, compila y graba **sin tocar ni una línea del programa**.

Mide otra vez las dos velocidades de parpadeo con el cronómetro y rellena esta tabla:

	PARPADEO CON HAL_DELAY	PARPADEO CONTANDO VUELTAS
A 84 MHz		
A 16 MHz		
¿Ha cambiado?		

Antes de mirar el resultado, apunta qué esperas que ocurra. Luego mídelo. **Si no coincide, esa diferencia es justo lo que hay que explicar en la memoria.**

> Cuando termines, **vuelve a dejar el proyecto a 84 MHz**: es la configuración con la que trabajan las siete prácticas de la asignatura.

Paso 7. Comprueba el reloj sin fiarte del código

En el paso anterior has deducido la frecuencia por el comportamiento del parpadeo. Ahora compruébala **mirándola**, que es distinto.

10. Lanza el programa en modo depuración y párala en cualquier punto.
11. Abre la vista de registros de periféricos y busca los del **RCC**.
12. Localiza los valores del PLL y comprueba que son los que dice tu configuración.

> **El registro no miente.** Si el código dice una cosa y el registro dice otra, la configuración no llegó a aplicarse, y ese es un problema muy distinto de un error de cálculo. Poder distinguir los dos casos en treinta segundos es la razón de aprender a usar esta vista.

En la memoria, indica **qué frecuencia has leído en el registro** y si coincide con la que esperabas.

6. Criterios de aceptación

Nº	PRUEBA	RESULTADO EXIGIDO
CA-1	Encender la placa con el programa grabado.	El LED verde parpadea una vez por segundo, sin el depurador conectado.
CA-2	Mantener pulsado el botón azul.	El LED sigue parpadeando a un ritmo **parecido**, mediante la espera contada.
CA-3	Quitar `volatile` del bucle de espera y compilar **en Release**.	El parpadeo con el botón pulsado se vuelve **invisible**: el compilador ha borrado el bucle. En **Debug** la misma prueba no cambia nada, y hay que saber decir por qué.

Nº	PRUEBA	RESULTADO EXIGIDO
CA-4	Volver a poner `volatile`. Cambiar el reloj a 16 MHz y regenerar.	El parpadeo con `HAL_Delay` **no cambia**; el de las vueltas se hace unas **cinco veces más lento**.
CA-5	Regenerar el código desde CubeMX sin cambiar nada.	El programa sigue estando ahí, porque se escribió dentro de los bloques USER CODE.
CA-6	Poner un punto de ruptura en el `HAL_GPIO_TogglePin` y depurar.	El depurador para en **esa línea de `main.c`**, no en ensamblador.

> CA-6 no es un adorno: es la comprobación de que la ruta del proyecto no tiene tildes. Si te salta a una ventana de ensamblador, revisa el aviso del Paso 1 antes de seguir con la Práctica 2.

7. Criterios de evaluación

> **Condición previa.** El proyecto debe compilar **sin errores y sin avisos**. Un proyecto que no compila no se evalúa: se devuelve para corrección y se califica sobre 5.

CRITERIO	QUÉ SE VALORA	ALCANZADO A MEDIAS	PESO
1. Proyecto y entorno	CA-1 y CA-6: el proyecto se crea, compila, graba y **depura**.	Compila y graba, pero el depurador no encuentra las fuentes.	1,5
2. Configuración de pines	PA5 salida y PC13 entrada, hechos desde el `.ioc` y con etiqueta.	Los pines funcionan pero se configuraron a mano en el código.	1,5

CRITERIO	QUÉ SE VALORA	ALCANZADO A MEDIAS	PESO
3. El contrato del generador	CA-5: todo el código propio dentro de los bloques USER CODE.	Hay código fuera de las marcas, aunque funcione.	2,0
4. Las dos esperas	CA-2 y CA-3: la espera contada funciona, y se entiende que `volatile` solo se nota con el optimizador activado.	Funciona, pero no sabe explicar por qué hace falta `volatile`, o cree que en Debug también se nota.	2,0
5. La medida del reloj	CA-4: la tabla del Paso 6, con las cuatro medidas tomadas.	La tabla está a medias o los valores no son creíbles.	2,0
6. Memoria	Las dos cuestiones del apartado 8.	Responde una de las dos, o sin justificar.	1,0

8. Entregables

- **Proyecto de STM32CubeIDE** comprimido, **sin las carpetas Debug ni Release**. Nombre: ACII_P1_Apellido1_Apellido2_Nombre.zip
- **La tabla del Paso 6** con las cuatro medidas y el número de vueltas al que llegaste.
- **Memoria breve**, máximo dos páginas:
    - Enumera los pines que CubeMX configuró sin que se lo pidieras al elegir la placa, y explica para qué sirve cada uno. ¿Qué pasaría si desactivaras los dos del depurador?
    - `HAL_Delay(500)` espera lo mismo a 16 que a 84 MHz. ¿Cómo puede ser, si el micro va cinco veces más despacio? Averigua de dónde saca `HAL_Delay` su base de tiempo y explícalo.

9. Uso de asistentes de inteligencia artificial

Se permite el uso de asistentes de IA como herramienta de consulta y de depuración, con tres condiciones:

- **Declararlo** en la memoria: qué asistente y para qué.
- **Entender lo que entregas.** En la defensa se te puede pedir que expliques o modifiques cualquier línea.
- **No sustituir la medida.** Los tiempos del Paso 6 los tomas tú, con la placa y un cronómetro.

Anexo A. Las claves del .ioc que conviene reconocer

CLAVE	QUÉ SIGNIFICA
`Mcu.CPN`	La referencia exacta del chip: `STM32F401RET6`.
`board`	La placa elegida. De aquí salen los pines que aparecen solos.
`Mcu.IPn`	Cada periférico activado: NVIC, RCC, SYS, USART2…
`Mcu.Pinn`	La lista de pines en uso.
`PA5.Signal`	Qué papel juega el pin: `GPIO_Output`, `GPIO_Input`, `USART2_TX`…
`PA5.GPIO_Label`	El nombre que tendrá en `main.h`. De aquí salen `LD2_Pin` y `LD2_GPIO_Port`.
`RCC.SYSCLKFreq_VALUE`	La frecuencia del núcleo, en hercios.
`RCC.PLLM`, `PLLN`, `PLLP`	Los tres divisores y el multiplicador que la producen.
`NVIC.*_IRQn`	Qué interrupciones están activadas.
`ProjectManager.KeepUserCode`	Si se conserva lo que hay entre las marcas USER CODE.
`ProjectManager.ProjectName`	El nombre del proyecto. Ojo: puede no coincidir con el de la carpeta.
`ProjectManager.TargetToolchain`	Para qué entorno se genera. Tiene que ser `STM32CubeIDE`: CubeMX propone `EWARM`, el de IAR.
`ProjectManager.UnderRoot`	Si el proyecto se genera en la raíz de la carpeta, que es donde lo busca la importación de CubeIDE. Tiene que valer `true`.

Anexo B. Qué hay en la carpeta del proyecto

RUTA	QUÉ CONTIENE	¿SE TOCA?
`Core/Src/main.c`	El programa. Aquí trabajas, dentro de las marcas.	sí
`Core/Inc/main.h`	Las definiciones de los pines.	no
`Core/Src/stm32f4xx_it.c`	Las funciones de atención a interrupciones.	dentro de las marcas
`Core/Src/stm32f4xx_hal_msp.c`	El arranque de reloj y pines de cada periférico.	no
`Core/Startup/`	El arranque en ensamblador y la tabla de vectores.	no
`Drivers/`	La biblioteca HAL de ST.	no
`*.ld`	Dónde se coloca cada cosa en memoria.	no
`*.ioc`	La configuración. La fuente de todo lo anterior.	sí, con CubeMX
`Debug/`, `Release/`	Resultado de compilar. **No se entrega.**	no

Anexo C. Conexionado

Esta práctica no necesita ningún componente externo: todo está ya en la placa.

SEÑAL	PIN	DETALLE
LED verde LD2	PA5	Activo a 1: se enciende escribiendo un uno.
Botón azul B1	PC13	Activo a 0: pulsado se lee `GPIO_PIN_RESET`.
Depurador SWD	PA13 / PA14	Los pone CubeMX solo. **No los desactives.**
Puerto serie	PA2 / PA3	USART2, conectado al ST-LINK. Aquí no se usa.